

Topic 2: Loops & Iterative Control Structures

1. High-Yield Conceptual Theory

1.1 Structural Taxonomy of Loops

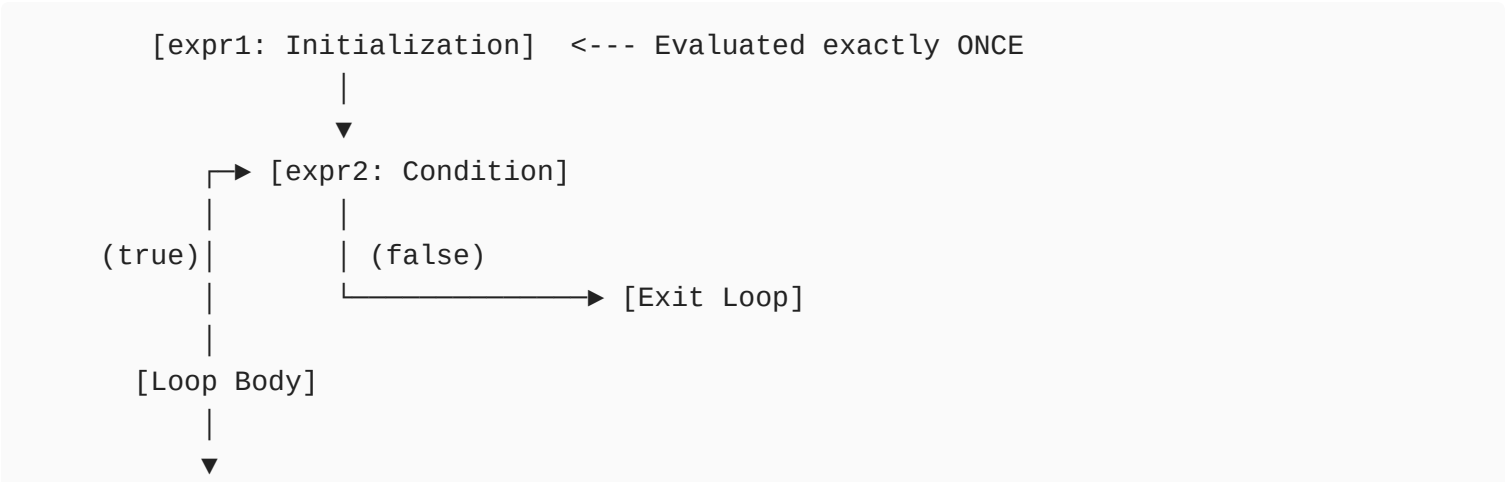
In C, loops are categorized into **entry-controlled** and **exit-controlled** based on where the loop condition is evaluated:

Feature	while Loop	for Loop	do-while Loop
Control Type	Entry-controlled	Entry-controlled	Exit-controlled
Minimum Iterations	0	0	1
Syntax	while (condition) { ... }	for (init; cond; update) { ... }	do { ... } while (condition);
Trailing Semicolon	No (treated as empty body)	No (treated as empty body)	Mandatory at the end of while(cond);

1.2 The for Loop Execution Lifecycle

A standard for loop is defined as:

```
for (expr1; expr2; expr3) {  
    body;  
}
```



[expr3: Update]



Clause Mechanics & Omission Rules

1. **expr1 (Initialization):** Evaluated only once before entering the loop. Multiple statements can be grouped using the comma operator (e.g., `for (i = 0, j = 10; ...; ...)`).
2. **expr2 (Loop Condition):** Evaluated **before each iteration**.
 - If omitted, the standard dictates it defaults to non-zero (`1` , i.e., **True**).
 - Hence, `for (;;)` creates an unconditional infinite loop.
 - If multiple expressions are combined using a comma operator:

```
for (i = 0, j = 0; i < 5, j < 10; i++, j++)
```

The condition evaluates `i < 5` , discards the result, and uses the truth value of `j < 10` alone to decide whether to continue.

3. **expr3 (Increment / Update):** Evaluated **after the loop body**, right before re-evaluating `expr2` .

1.3 Loop Control Statements: `break` vs. `continue`

Both keywords modify the normal linear sequence of loop execution:

- **`break`** : Terminates the innermost enclosing loop (`for` , `while` , `do-while`) or `switch` statement immediately. Control passes to the statement immediately following that block.
- **`continue`** : Skips the rest of the current iteration body.

Critical Semantic Difference between `for` and `while` upon `continue`

This is one of the most frequently exploited traps in GATE:

- **In a `while` / `do-while` loop:**
Executing `continue` jumps **directly to the condition check**. If the loop increment/update statement resides inside the loop body after the `continue` , it is **bypassed**, frequently leading to an **infinite loop**.

```
int i = 0;
while (i < 5) {
    if (i == 2)
        continue; // Jumps directly to (i < 5); i++ is skipped -> INFINITE LOOP
```

```
    i++;  
}
```

- **In a for loop:**

Executing `continue` jumps to **expr3 (the update expression)** first, evaluates it, and **then** proceeds to `expr2` (the condition).

```
for (int i = 0; i < 5; i++) {  
    if (i == 2)  
        continue; // Jumps to i++, then checks i < 5 -> Terminates normally  
}
```

1.4 Postfix vs. Prefix in Loop Conditions

The most pervasive myth regarding postfix decrement (`x--`) in a loop condition is that *the decrement is delayed until the end of the loop body*.

In C, the end of the controlling expression in a `while` , `for` , or `do-while` statement is a **sequence point** (C99/C11: C23 models this as "sequenced before"). All side effects of the condition—including postfix updates—are guaranteed to complete **before the first statement in the loop body executes**.

Mechanics & Comparative Execution Trace

Consider initial value $x = 3$:

Step	<code>while (x-- > 0)</code>	<code>while (--x > 0)</code>
Iter 1 Test	Evaluates $3 > 0$ (true); $x \rightarrow 2$	$x \rightarrow 2$; evaluates $2 > 0$ (true)
Iter 1 Body	Body executes with $x = 2$	Body executes with $x = 2$
Iter 2 Test	Evaluates $2 > 0$ (true); $x \rightarrow 1$	$x \rightarrow 1$; evaluates $1 > 0$ (true)
Iter 2 Body	Body executes with $x = 1$	Body executes with $x = 1$
Iter 3 Test	Evaluates $1 > 0$ (true); $x \rightarrow 0$	$x \rightarrow 0$; evaluates $0 > 0$ (false) $\rightarrow$ Exit
Iter 3 Body	Body executes with $x = 0$	<i>Does not execute</i>
Exit Test	Evaluates $0 > 0$ (false); $x \rightarrow -1 \rightarrow$ Exit	<i>(Exited on Iter 3 Test)</i>
Total Runs	3 iterations (N times)	2 iterations ($N - 1$ times)
Value Inside Body	$N - 1, N - 2, \dots, 0$	$N - 1, N - 2, \dots, 1$
Final Exit State	$x = -1$	$x = 0$

Pitfall 1: Post-Loop Value Drift & Off-by-One Exit State

Because postfix decrements *even when the condition fails*, x drops below the lower bound by 1 upon loop termination.

```
#include <stdio.h>

int main(void) {
    int x = 5;
    while (x-- > 0) {
        // Body runs 5 times
    }

    // Common assumption: x == 0
    // Actual state: x == -1
    printf("Post-loop x: %d\n", x); // Output: -1

    int arr[5] = {10, 20, 30, 40, 50};
    // arr[x] here causes an illegal out-of-bounds access: arr[-1]!
    return 0;
}
```

If the post-loop code checks `if (x == 0)` or reuses `x` as an index, it triggers logic bugs or memory corruption. With prefix `(--x > 0)`, the loop exits with `x == 0`.

Pitfall 2: The unsigned Postfix Underflow Trap

Replacing signed `int` with an unsigned type creates two distinct failure modes depending on whether the comparison operator is `>` or `>=`.

Trap A: The Infinite Loop with `>=`

```
unsigned int n = 5;

// BUG: Infinite loop!
while (n-- >= 0) {
    printf("%u\n", n);
}
```

- An unsigned `int` is mathematically *always* ≥ 0 .
- When $n = 0$, $0 \geq 0$ is **true**. n decrements to $0 - 1 = \text{UINT_MAX}$ (4,294,967,295), and the loop never halts.

Trap B: The Hidden Wrap-Around with `>`

```
unsigned int n = 3;
```

```

while (n-- > 0) {
    // Runs 3 times: n = 2, 1, 0
}

// Loop terminates when n is tested at 0 (0 > 0 is false).
// BUT postfix still decrements n!
// 0 - 1 wraps around to UINT_MAX.
printf("Final n: %u\n", n); // Output: 4294967295 (UINT_MAX)

```

- The loop body terminates correctly, but `n` does **not** exit at `0` —it exits at `UINT_MAX`. Any subsequent arithmetic like `n + 1` or buffer bounds checking relying on `n` will fail.

Pitfall 3: Array Traversal Off-by-One (`arr[x]`)

A common idiom for reverse array traversal is `while (i-- > 0)`. The programmer often mistakenly writes `arr[i]` expecting the initial value of `i`.

```

#include <stdio.h>

int main(void) {
    int arr[3] = {10, 20, 30};
    int i = 3;

    // Correct idiom for zero-indexed arrays:
    // Iter 1: tests 3 > 0, i becomes 2 -> arr[2] = 30
    // Iter 2: tests 2 > 0, i becomes 1 -> arr[1] = 20
    // Iter 3: tests 1 > 0, i becomes 0 -> arr[0] = 10
    // Terminate: tests 0 > 0, i becomes -1
    while (i-- > 0) {
        printf("%d ", arr[i]); // Output: 30 20 10
    }

    // PITFALL: Using prefix --i
    int j = 3;
    // Iter 1: j becomes 2, tests 2 > 0 -> arr[2] = 30
    // Iter 2: j becomes 1, tests 1 > 0 -> arr[1] = 20
    // Iter 3: j becomes 0, tests 0 > 0 (false) -> Skips arr[0] entirely!
    while (--j > 0) {
        printf("%d ", arr[j]); // Output: 30 20 (arr[0] missed!)
    }
    return 0;
}

```

Pitfall 4: `do { ... } while (x-- > 0)` Execution Inversion

In a `do-while` loop, the body runs **before** the first condition test and decrement.

```
#include <stdio.h>

int main(void) {
    int x = 0;

    // Body runs ONCE before condition is evaluated
    do {
        printf("Body x = %d\n", x); // Prints: Body x = 0
    } while (x-- > 0);
    // Condition test: evaluates 0 > 0 (false).
    // Postfix still fires: x becomes -1.

    printf("Exit x = %d\n", x);    // Prints: Exit x = -1
    return 0;
}
```

If $x = 1$:

1. Body runs with $x = 1$.
2. Condition tests $1 > 0$ (**true**); x decrements to 0.
3. Body runs second time with $x = 0$.
4. Condition tests $0 > 0$ (**false**); x decrements to -1 .
5. Body ran 2 **times** for initial $x = 1$ (compared to 1 time in a standard `while`).

Pitfall 5: Macro Expansion Double Side-Effects

Wrapping conditions containing mutating operators in macros without parameter caching causes multiple decrements per check.

```
#include <stdio.h>

#define IS_POSITIVE(val) (((val) > 0) && ((val) < 100))

int main(void) {
    int x = 2;

    // Macro expands to: (((x--) > 0) && ((x--) < 100))
    // x is decremented TWICE in a single iteration!
    while (IS_POSITIVE(x--)) {
        printf("x = %d\n", x);
    }
    return 0;
}
```

Quick Comparison Matrix

Expression	Condition Tested	Value Inside Body (Iter 1)	Total Iterations	Final Value After Loop
<code>while (x-- > 0)</code>	$x_{\text{old}} > 0$	$N - 1$	N	-1
<code>while (--x > 0)</code>	$(x_{\text{old}} - 1) > 0$	$N - 1$	$N - 1$	0
<code>while (x-- >= 0)</code> <i>(signed)</i>	$x_{\text{old}} \geq 0$	$N - 1$	$N + 1$	-1
<code>while (x-- >= 0)</code> <i>(unsigned)</i>	$x_{\text{old}} \geq 0$	$N - 1$	∞ (Infinite)	Never exits
<code>do { ... } while (x-- > 0)</code>	$x_{\text{old}} > 0$	N (first run)	$N + 1$	-1

1.5 Mathematical Analysis of Loop Iterations

GATE frequently requires calculating the exact number of times a statement executes or finding the asymptotic time complexity $\Theta(g(n))$:

Loop Pattern	Step Behavior	Total Iterations	Asymptotic Complexity
<code>for (i = 1; i <= n; i += c)</code>	Arithmetic Progression	$\lfloor \frac{n-1}{c} \rfloor + 1$	$\Theta(n)$
<code>for (i = 1; i <= n; i *= c) (c > 1)</code>	Geometric Progression	$\lfloor \log_c n \rfloor + 1$	$\Theta(\log n)$
<code>for (i = n; i >= 1; i /= c) (c > 1)</code>	Geometric Progression	$\lfloor \log_c n \rfloor + 1$	$\Theta(\log n)$
<code>for (i = 2; i <= n; i = i * i)</code>	Double Exponential Step	$\lfloor \log_2(\log_2 n) \rfloor + 1$	$\Theta(\log \log n)$
<code>for (i = 1; i * i <= n; i++)</code>	Quadratic Bound	$\lfloor \sqrt{n} \rfloor$	$\Theta(\sqrt{n})$
<code>for (i = 1; i <= n; i++) for (j = 1; j <= i; j++)</code>	Triangular Sum	$\sum_{i=1}^n i = \frac{n(n+1)}{2}$	$\Theta(n^2)$
<code>for (i = 1; i <= n; i++) for (j = 1; j <= n; j += i)</code>	Harmonic Sum	$\sum_{i=1}^n \frac{n}{i} = n \sum_{i=1}^n \frac{1}{i}$	$\Theta(n \log n)$

2. Common GATE Pitfalls & Edge Cases

1. The Accidental Semicolon Trap

A semicolon directly after a loop header creates an empty null statement as the loop body:

```
int i = 0;
while (i < 5); // Infinite loop! i is never incremented.
{
    i++;
}
```

For `for` loops:

```
int i;
for (i = 0; i < 5; i++);
printf("%d", i); // Prints 5
```

2. Unsigned Integer Underflow in Loop Condition

When using unsigned data types as decrementing loop counters:

```
for (unsigned int i = 10; i >= 0; i--) {
    // Infinite loop!
}
```

- **Explanation:** An `unsigned int` can never be negative. When `i = 0`, `i--` wraps around to $2^{32} - 1$ (or $2^{16} - 1$), which is always ≥ 0 .

3. Floating-Point Equality & Step Distortion

```
for (float x = 0.0; x != 1.0; x += 0.1) {
    // Likely infinite loop!
}
```

- **Explanation:** Binary floating-point cannot represent 0.1 exactly. Successive additions introduce rounding errors, so `x` may jump from 0.999999 to 1.099999, never strictly equaling 1.0.

4. Dependent Comma Operations in Condition

```
int i = 0, j = 5;
while (i < 3, j > 2) {
    i++;
    j -= 2;
}
```

- The expression $i < 3$, $j > 2$ evaluates $i < 3$, discards it, and branches solely on $j > 2$. When j reaches 1, the loop terminates even if i is still < 3 .
-

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2017)

Question:

Consider the following C program:

```
#include <stdio.h>
int main() {
    int m = 10;
    int n, n1;
    n = ++m;
    n1 = m++;
    n--;
    --n1;
    n -= n1;
    printf("%d", n);
    return 0;
}
```

What is the value printed by the program?

Step-by-step Solution:

1. $m = 10$
 2. $n = ++m$; : Pre-increment $m \rightarrow m = 11$, then assign $\rightarrow n = 11$.
 3. $n1 = m++$; : Post-increment $m \rightarrow n1$ gets original value 11, then m becomes 12.
 4. $n--$; : Post-decrement $n \rightarrow n$ becomes $11 - 1 = 10$.
 5. $--n1$; : Pre-decrement $n1 \rightarrow n1$ becomes $11 - 1 = 10$.
 6. $n -= n1$; : $n = n - n1 = 10 - 10 = 0$.
 7. Output printed: 0.
-

PYQ 2 (GATE CS 2015, Set 1)

Question:

Consider the following C function:

```

int fun1(int n) {
    static int i = 0;
    if (n > 0) {
        ++i;
        fun1(n - 1);
    }
    return (i);
}

int fun2(int n) {
    static int i = 0;
    if (n > 0) {
        i += fun1(n);
        fun2(n - 1);
    }
    return (i);
}

int main() {
    printf("%d", fun2(5));
    return 0;
}

```

What does the program print?

Step-by-step Solution:

- Notice that `i` in `fun1` is static (persists across calls to `fun1`), and `i` in `fun2` is static (persists across calls to `fun2`).
- Let us trace `fun2(5)` :
 - For $n = 5$:
 - Calls `fun1(5)` . In `fun1` : n goes 5, 4, 3, 2, 1. `++i` runs 5 times. `fun1` returns 5 .
 - In `fun2` : $i += 5 \rightarrow i$ in `fun2` is 5.
 - Calls `fun2(4)` .
 - For $n = 4$:
 - Calls `fun1(4)` . In `fun1` : static `i` was already 5. n goes 4, 3, 2, 1. `++i` runs 4 more times $\rightarrow i = 5 + 4 = 9$.
 - `fun1` returns 9 .
 - In `fun2` : $i += 9 \rightarrow i$ in `fun2` is $5 + 9 = 14$.
 - Calls `fun2(3)` .
 - For $n = 3$:
 - Calls `fun1(3)` . `++i` in `fun1` runs 3 times $\rightarrow i = 9 + 3 = 12$.
 - `fun1` returns 12 .
 - In `fun2` : $i += 12 \rightarrow i = 14 + 12 = 26$.

- Calls `fun2(2)` .
- For $n = 2$:
 - Calls `fun1(2)` . `++i` in `fun1` runs 2 times $\rightarrow i = 12 + 2 = 14$.
 - `fun1` returns 14 .
 - In `fun2`: `i += 14` $\rightarrow i = 26 + 14 = 40$.
 - Calls `fun2(1)` .
- For $n = 1$:
 - Calls `fun1(1)` . `++i` in `fun1` runs 1 time $\rightarrow i = 14 + 1 = 15$.
 - `fun1` returns 15 .
 - In `fun2`: `i += 15` $\rightarrow i = 40 + 15 = 55$.
 - Calls `fun2(0)` .
- For $n = 0$: `if (n > 0)` is false. Returns `i` from `fun2` , which is 55 .

Output: 55

PYQ 3 (GATE CS 2014)

Question:

Consider the following C function:

```
int f(int j) {
    static int s = 1;
    int k;
    for (k = 1; k < j; k++)
        s = s + 1;
    return s;
}
```

If the function is called as:

```
for (i = 1; i <= 5; i++)
    sum = sum + f(i);
```

Assuming `sum` is initialized to 0, what is the final value of `sum` ?

Step-by-step Solution:

`s` is `static` , so its value carries over between calls to `f` . Initially $s = 1$.

- **Call 1 ($i = 1$):** `f(1)`
 - Inner loop: `for (k = 1; k < 1; k++)` $\rightarrow$ Condition $1 < 1$ is false. Loop executes 0 times.

- Returns $s = 1$.
- `sum = 0 + 1 = 1`.
- **Call 2 (i = 2): f(2)**
 - Inner loop: $k = 1; k < 2 \rightarrow$ Runs for $k = 1$ (1 time).
 - $s = s + 1 = 1 + 1 = 2$.
 - Returns $s = 2$.
 - `sum = 1 + 2 = 3`.
- **Call 3 (i = 3): f(3)**
 - Inner loop: $k = 1, 2$ (2 times).
 - $s = 2 + 2 = 4$.
 - Returns $s = 4$.
 - `sum = 3 + 4 = 7`.
- **Call 4 (i = 4): f(4)**
 - Inner loop: $k = 1, 2, 3$ (3 times).
 - $s = 4 + 3 = 7$.
 - Returns $s = 7$.
 - `sum = 7 + 7 = 14`.
- **Call 5 (i = 5): f(5)**
 - Inner loop: $k = 1, 2, 3, 4$ (4 times).
 - $s = 7 + 4 = 11$.
 - Returns $s = 11$.
 - `sum = 14 + 11 = 25`.

Output: 25

4. Tricky Practice Questions (Edge Cases & Traps)

Practice Question 1: Modification of Loop Counter Inside Body

Question:

What will be the output of the following C program?

```
#include <stdio.h>
int main() {
    int count = 0;
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 0) {
            i += 2;
        }
        count++;
    }
    printf("%d", count);
    return 0;
}
```

```
}
```

Detailed Explanation:

Trace step-by-step:

1. $i = 0$: $0 < 10$ (true). $0 \% 2 == 0$ is true $\rightarrow i = 0 + 2 = 2$. $count = 1$. Loop update executes: $i++ \rightarrow i = 3$.
2. $i = 3$: $3 < 10$ (true). $3 \% 2 == 0$ is false. $count = 2$. Loop update executes: $i++ \rightarrow i = 4$.
3. $i = 4$: $4 < 10$ (true). $4 \% 2 == 0$ is true $\rightarrow i = 4 + 2 = 6$. $count = 3$. Loop update: $i++ \rightarrow i = 7$.
4. $i = 7$: $7 < 10$ (true). $7 \% 2 == 0$ is false. $count = 4$. Loop update: $i++ \rightarrow i = 8$.
5. $i = 8$: $8 < 10$ (true). $8 \% 2 == 0$ is true $\rightarrow i = 8 + 2 = 10$. $count = 5$. Loop update: $i++ \rightarrow i = 11$.
6. $i = 11$: $11 < 10$ is false. Loop terminates.

Output: 5

Practice Question 2: do-while with Decrementing Postfix

Question:

What is the exact output printed by the following code?

```
#include <stdio.h>
int main() {
    int i = 5;
    do {
        printf("%d ", i);
    } while (--i > 2);
    printf("| Final: %d", i);
    return 0;
}
```

Detailed Explanation:

1. $i = 5$: Body prints 5 .
 - Check: $--i > 2 \rightarrow$ pre-decrement i to 4; test $4 > 2$ (true).
2. $i = 4$: Body prints 4 .
 - Check: $--i > 2 \rightarrow$ pre-decrement i to 3; test $3 > 2$ (true).
3. $i = 3$: Body prints 3 .
 - Check: $--i > 2 \rightarrow$ pre-decrement i to 2; test $2 > 2$ (false).
4. Loop exits. Final `printf` outputs current value of i , which is 2 .

Practice Question 3: Comma Operator in while Condition

Question:

How many times is the statement inside the loop executed?

```
#include <stdio.h>
int main() {
    int a = 5, b = 0;
    while (a = 0, b < 3) {
        b++;
    }
    return 0;
}
```

Detailed Explanation:

1. The condition is `(a = 0, b < 3)`.
2. The comma operator evaluates `a = 0` (which assigns 0 to `a`), discards the value 0, and returns the result of `b < 3`.
3. Iterations:
 - Iteration 1: $b = 0 < 3$ (true) $\rightarrow b = 1$.
 - Iteration 2: $b = 1 < 3$ (true) $\rightarrow b = 2$.
 - Iteration 3: $b = 2 < 3$ (true) $\rightarrow b = 3$.
 - Iteration 4 check: $b = 3 < 3$ (false). Loop terminates.
4. Total executions of `b++` = 3.

Output: 3

5. Quick Revision & Edge-Case Checklist

- ☐ `while (condition);` with a semicolon executes an empty statement repeatedly until the condition fails. If the condition depends on variables modified inside `{ }`, it hangs.
- ☐ In `for (init; cond; update)`, `continue` triggers `update` before checking `cond`. In `while`, `continue` jumps immediately to `cond`.
- ☐ An unsigned loop counter with condition `i >= 0` causes an infinite loop due to modular wrap-around ($0 - 1 = \text{UINT_MAX}$).
- ☐ `for (;;)` condition defaults to true (`1`), creating an infinite loop.

- ☐ In comma-separated loop conditions `for (...; c1, c2; ...)`, only `c2` governs continuation; `c1` is discarded.
 - ☐ Avoid exact equality (`!=` or `==`) checks on `float` or `double` loop variables.
-